Probabilistic Programming (Probability Monad)

We are building a model of a simplified Poker game, just to give you an idea how does one interact with a probabilistic programming framework. In order, to allow you to gain some more confidence in handling Scala, we are minimizing the template files this week. The file `Poker.scala` contains just a minimal skeleton, and the build script is set up. The exercises will guide to build a simple reasoning system about poker.

A basic tutorial on building discrete models in Probula is available in the libraries repository:

We are going to use the Probula library (https://wasowski.codeberg.page/probula/), developed for the purpose of this course. Resources available:

- Lecture slides
- Source code https://codeberg.org/wasowski/probula
- A basic tutorial in the code base of Probula https://codeberg.org/wasowski/probula/src/branch/main/doc/tutorial/Basic.sc
- Scaladoc: https://wasowski.codeberg.page/probula/api/index.html

It is highly recommended to work through the tutorial in `Basic.sc` first (see above for the link). You can open scala-cli console in the directory of this exercise and execute it line-by-line.

For many exercises, you are (quietely) expected to run the queries in the REPL in order to inspect probability values (or to print them to the standard output).

Credits. The exercise has been originally inspired by the material of a probabilistic programming course by prof. Joost-Pieter Katoen at RWTH Aachen, Germany.

Hand in: `Exercises.scala`.

Peter and Paula play a game. An urn contains some black balls and a single red ball. They take turns taking one random ball out of the urn. The first one to pick the red ball wins. We model the players using an algebraic data type, the enumeration type `Player` (follow in the `Exercises.scala`). We add a function `next` to determine the next player, following a simple alternation. The function is in `Exercises.scala`.

Exercise 1. Write a function `pick` that given the number `n` of black balls in the urn returns a probability distribution `Dist[Ball]` that generates Red with the probability of picking the red ball. There is always one red ball inside the urn, when a move is made (when this one is picked, the game ends). Use the following Probula constructor to implement `pick`.

```
Probula.bernoulli[U](p:Prob, success:U, failure:U):Dist[U]
```

You will need to calculate the probability of success from `n`. Remember the `.pr` extension method that turns a `Double` number into a probability value, for instance `0.5.pr:Prob`.

Exercise 2. Write a function `move` that given the starting player and the number of black balls `n` present in the urn returns the probability distribution defining which player wins the entire game (which player picks the red ball first). *Hint:* My solution used `pick` and `dirac`.

The univariate version is: `Dirac[A](a:A):Dist[A]`

The "fluent" combinator is: `(_ : Dist[U]).dirac[A](a:A):Dist[Tuple.Append[U,A]]`,

where `U` is a tuple (it also works for a scalar).

The `Dirac` constructor returns a distribution where the value `a` has probability `1`. Dirac is basically a

constant (non-probabilistic) probability distribution—only one outcome is legal. This is the monadic const, unit, or pure.

Exercise 3. Peter is polite and lets Paula decide who starts: herself or him. Use the function `move` to estimate the chance of Paula winning when she starts, and when Peter starts, assuming that the urn contains `BallsNo` balls in total (including one red).

To calculate probability, first use `(_:Dist[T]).sample(N)` for some object of type `Dist[...]` to obtain a sample of `N` elements. Then use the `IData`'s method `(_:IData[T]).pr(...)` to calculate the probability. The latter takes a value from the distribution range. It returns the estimate of probability that the distribution takes this value.

Which strategy is better for Paula with `BallsNo==8`? What if `BallsNo==9`?

Exercise 4. A pen-and-pencil question: Estimate the size of the Bayesian network generated by `move (p, 10)` for some player constant p? (so how many random variables are derived?)

Observe, that this model would be very annoying and laborious to build manually on paper, but with statistical interpretation in a programming framework we can build models for 200 balls easily.

This question is not graded. Use it yourself to appreciate the power of the probabilistic programming.

Exercise 5. Now assume that we know that Paula won the game. What is the probability that she has started the game? The task is split in a number of smaller steps.

We first create a bivariate model that captures the joint-distribution of the starting-player and the winning player. The first component can be just uniform prior that says Paula and Peter are equally likely to start (`Dist[Player]`). The second component describes the random variable showing the winner for a game played with `BallsNo` balls in the urn using the `move` function defined earlier. We want to keep both the first mover and the winner in the model, so that we can reason about who has started under the condition of who has won. Thus the type `Dist[(Player, Player)]` seems appropriate for the entire model. Here is the structure of the model mathematically:

$$\begin{aligned} \text{player-1} &\sim \mathcal{U}(\text{Paula}, \text{Peter}) \\ \text{winner} &\sim \text{move}(\text{player-1}, \text{BallsNo} - 1) \end{aligned}$$

The `probDep` method (a simple wrapper for `flatMap` which does not override the source varible) may prove useful. Sadly, type inference often fails with `probDep`. I recommend adding a type annotation to the lambda expression in the argument, e.g. `probDep { (n:Int) =>... }`

I recommend trying both a model with for-yield and with the fluent API (they are both nice).

Now that this model is built, compute the probability that Paula wins if there is an equal probability that either of them started the game.

Finally, compute the probability that Paula has started given that she has won.This is done most easily by filtering/matching the condition that Paula is the winner, sampling, and computing the probability that she started by another match. Does the result agree with your intuition? (cf. Exercise 3)

Exercise 6. We know that winning player wins approximately 1/2 games when she starts, and we know now that if there is an even number of balls in the urn then the probability is precisely equal for both players, while if the number of balls is odd the probability of the first player winning is slightly higher.

In this exercise, we assume that the number of balls is unknown, but it is taken from range 1 to 6 with uniform probability (uniform prior) and we will observe that Player-1 has won. We will ask what is the probability that the urn held an odd number of balls in the beginning of the game. We expect this probability to be slightly higher than 50%, as Player-1 winning makes as believe slightly that an odd number of balls are in the urn.

Let `UpperBound` will be the maximum number of balls in the urn that we consider. Then $\mathtt{n} \in [1, \text{UpperBound})$ is the number of black balls. We build a bivariate model, where the first component describes the number of balls in the urn, the second one describes the chance of Paula winning. The model also assume that Paula is Player-1 (without loss of generality).

$$\begin{aligned} \mathtt{n} &\sim \mathcal{U}(1..\text{UpperBound}) \\ \text{winner} &\sim \text{move}(\text{Paula}, \mathtt{n}) \\ \text{winner} &= \text{Paula} \end{aligned}$$

Finally, compute the posterior probability `posteriorODd` that the number of balls was odd. This can be done using the `(_:IData[T]).pr` method. We need a slightly different version of the former that takes a predicate, not a concrete value (or the pattern match version):

```
(_:IData[T]).pr (p:T =>Boolean)
(_:IData[T]).prMatching { case ... =>}
```

Is `posteriorOdd` greater than 1/2? Why? Would you prefer to answer these kind of questions by performing the analysis on paper instead of programming?